

From Open Questions to a Verifiable Synthesis Architecture

Answers to Leant2's Further-Improvements Proposal

Prepared for Vladimir Reshetnikov

September 21, 2026

Scope. All 31 questions in R1–R9 of the proposal in `docs/proposals/11-further-improvements`, together with the engineering changes needed to make the answers implementable.

Repository snapshot.

784664ff34e819422510a4d470ab07fe48bb736b

Evidence status. Source and literature investigation, mathematical arguments, and executable finite-model checks. No Leant2 performance measurements or Lean compilation were performed for this report. Proposed improvements are not presented as implemented features.

Abstract

This report answers the expert questions in Leant2's September 2026 improvement plan and converts them into a staged architecture. Several uncertainties have useful published answers: Smyth supports higher-order examples and does not require trace-complete examples; Hoogle+ explicitly treats type classes by dictionary translation; Canonical demonstrates indexed recursion and generalized induction; AutoLifter addresses auxiliary-state synthesis; and container-based realizability analysis gives a principled starting point for polymorphic observations. Other questions require more carefully delimited guarantees. We separate correctness of accepted programs, preservation of solutions during pruning, bounded completeness, ranking optimality, and reproducibility. We give a constraint-component search model, admissible cost bounds, a sound finite-carrier exclusion test, typed and versioned observation caches, a conservative path to evaluation with holes, bounded motive enumeration, and a proof-producing realization interface for recursion. Counterexamples correct several tempting inferences: reverse is an ordinary fold into lists, the number of open goals is not an admissible general heuristic, and sorting concurrent results does not make a wall-clock search machine-independent. The final sections specify integration points, validation obligations, and an experimental plan without inventing speedups or model-accuracy figures.

Contents

1	Scope, evidence, and the guarantees that matter	3
1.1	Five guarantees, not one	3
1.2	A source-level integration observation	4
1.3	Corrections that change the design	4
2	R1: search control with coupled goals	5
2.1	R1.1. Can coupled goals be organized without wholesale copying or sequentialization?	5
2.2	R1.2. What admissible heuristic survives shared metavariables?	6
2.3	R1.3. Does just-in-time learning preserve meaningful ranking?	7
2.4	R1.4. Can a wall-clock search be machine-speed independent?	7
3	R2: carriers, accumulators, and auxiliary state	8
3.1	R2.1. Can finite examples determine a minimal carrier type?	8
3.2	R2.2. What replaces the alphabet for abstract polymorphic elements?	10
3.3	R2.3. Can fusion be run backwards to invent auxiliary information?	10
3.4	R2.4. Is there a complete motive enumeration for primitive recursion with parameters?	11
3.5	R2.5. Do generalization and lemma-speculation heuristics transfer?	12
4	R3: evaluation of programs with holes	12
4.1	R3.1. Has live bidirectional evaluation reached higher-order examples or dependent types?	12
4.2	R3.2. What incremental normalizer can support assignment and rollback?	13
4.3	R3.3. What is the cheapest sound fast evaluator?	14
4.4	R3.4. Can top-down angelic recursion have termination and completeness guarantees?	15
5	R4: indexed families, motives, and transport	15
5.1	R4.1. Is maximal-occurrence motive enumeration finite and complete?	15
5.2	R4.2. Can transports be canonicalized with setoids or heterogeneous equality?	16
5.3	R4.3. How do Canonical and sauto manage unfolding?	17
5.4	R4.4. Which Lean motive-computation functions are reusable?	17
6	R5: recursion capabilities and realization	18
6.1	R5.1. Does search-time evaluation agree definitionally with realized recursion?	18
6.2	R5.2. Does angelic execution compose with top-down search?	19
6.3	R5.3. Is there a canonical small basis with measured textbook coverage?	20

7	R6: contracts beyond finite decidable observations	21
7.1	R6.1. How should value and proof holes be interleaved without an SMT solver?	21
7.2	R6.2. Which properties follow by induction, simplification, and arithmetic? . . .	21
7.3	R6.3. Which automated-induction facilities are usable in Lean?	22
7.4	R6.4. Is there a principled proof-debt schedule?	23
8	R7: library retrieval and type-guided refinement	23
8.1	R7.1. How can dependent and universe-polymorphic types be abstracted usefully?	23
8.2	R7.2. Has type-guided refinement handled type classes already?	24
8.3	R7.3. Can Lean’s selection interface accept data goals and examples?	25
9	R8: meaningful results and equivalence	26
9.1	R8.1. What ranking objective is defensible?	26
9.2	R8.2. What inexpensive equivalence is safe for dependent programs?	27
10	R9: model-guided proposals with independent certification	27
10.1	R9.1. What accuracy do current models achieve on carrier and helper proposals?	27
10.2	R9.2. What makes model-assisted search deterministic and auditable?	28
11	An integrated implementation plan	29
11.1	Keep the semantic interfaces narrow	29
11.2	Implement in dependency order	30
11.3	Concrete source integration points	31
11.4	Experiments that can falsify the recommendations	32
11.5	What should count as a successful next release?	33
A	Coverage index: all 31 expert questions	34
B	Executable checks and the limits of the evidence	34
B.1	Reproduction	35
B.2	What was not established	36
	References	37

1 Scope, evidence, and the guarantees that matter

The starting point is the repository’s further-improvements proposal, not an imagined future implementation. The questions occur in eight source files: R1–R7 each have their own section file, while both R8 and R9 occur in `12-ranking.tex`. There are 31 numbered questions in total. This report keeps their R-number and local question number so every answer is traceable. The coverage index in Appendix A also gives page references.

The inspected repository revision is `784664ff34e819422510a4d470ab07fe48bb736b`. The proposal’s measurements refer instead to `33cec8b`, Lean 4.34.0, and a Windows workstation. In particular, its 278/278 baseline score, 13 unsolved Church stretch cases, and residual-evaluation profile are *reported measurements*, not new measurements made here [1]. The source of `Query` has a 20,000 ms default, while several documented experiments use 5, 6, or 10 seconds. Those are different configurations, not contradictory results [2].

The investigation examined the proposal, its bibliography, the acceptance gate and core result types, the transaction layer, and relevant portions of the engine and search implementation. For implementation-specific answers, selected Lean files were checked at tag `v4.34.0`. The rolling Lean reference consulted on September 21 describes 4.35.0-rc2; claims drawn from it are not silently attributed to the older toolchain [7, 8, 9].

1.1 Five guarantees, not one

Let a frozen query consist of an environment E , a target T , a contract $C : T \rightarrow \text{Prop}$, and an allowed-axiom policy A . Let G_B be the explicitly bounded candidate grammar. The following distinctions should appear in both implementation documentation and user-facing receipts.

Acceptance correctness. An accepted result includes closed expressions $p : T$ and $\pi : C(p)$, checked in E , with their transitive axiom dependencies permitted by A . This is a statement about a particular program and proposition. It says neither that the proposition fully captures the user’s intent nor that a separately compiled implementation has been proved equivalent.

Pruning correctness. Deleting a search state must not delete a completion satisfying the contract. If s represents a partial program and $\text{Comp}(s)$ its possible completions, a sound refutation establishes

$$\forall p \in \text{Comp}(s), \quad \neg C(p).$$

A final kernel check cannot repair a valid solution that was discarded earlier. Unsound pruning may preserve correctness of every accepted answer while destroying completeness.

Bounded completeness. Every satisfying member of G_B is eventually considered, or at least one is found whenever one exists, depending on the advertised service. This needs a finite, fully specified grammar, fair exploration, and correct rejection decisions. An unresolved type-checking or proof-search timeout is not evidence that an alternative is impossible.

Ranking optimality. The returned candidate minimizes a stated, fixed objective over a stated candidate set. A heuristic search priority, a term-size ranking, and a display preference are not automatically the same objective. An anytime answer need not be optimal even when every term in it is certified.

Reproducibility. Proof replay, search replay, and reproduction of a model’s inference are three distinct tasks. Rechecking the same closed proof can be reproducible even when another run discovers a different proof. Identical search events require a fixed environment, policy, proposal stream, resource accounting, and tie-breaking discipline.

The repository already embodies much of the acceptance discipline: `gate` instantiates terms,

rejects unresolved holes and escaped locals, performs synchronous kernel checking, and audits axiom dependencies [3]. These checks are a good foundation. The remaining guarantees require additional invariants outside the gate.

1.2 A source-level integration observation

The present gate accepts an optional pair consisting of a proof and its claimed type. It does not itself receive the original contract. In the inspected engine, the caller correctly constructs that type as $C(p)$ from the synthesized subtype value. Thus the observation is *not* a demonstrated soundness failure in the current pipeline. It is an interface-hardening opportunity: a future public gate should receive the frozen query and derive the expected program type and proof type itself. Receipts should preserve the original target and contract, rather than only a caller-supplied proof expression [3, 2, 4].

Similarly, `Accepted` currently records the program, its type, universes, proof, axioms, and classical flag, but not the original contract. The implemented `Outcome` has no explicit “unrefuted but uncertified” constructor. The proposal’s intended richer outcome algebra should therefore be implemented, not assumed already present [4].

1.3 Corrections that change the design

The most consequential corrections are summarized here; the detailed arguments appear in the answers.

Topic	Correction and consequence
A* lower bounds	Counting open goals can overestimate remaining cost when exact moves cost zero or one assignment closes several goals. Use a proved relaxation, not the count alone.
Carrier expressibility	Reverse is a right fold into <code>List A</code> ; a continuation carrier improves the operational form. Failure at a small search depth is not semantic impossibility.
Prior auxiliary-state work	AutoLifter directly synthesizes auxiliary information and combinators. The unsolved part is adapting such ideas to sparse contracts and Lean-dependent carriers, not the existence of auxiliary-state synthesis as a research topic.
Live evaluation	Smyth’s implementation supports higher-order examples and removes trace completeness. Dependent typed closures remain a separate adaptation problem.
Indexed proof search	Canonical already demonstrates indexed recursion and a generalized-induction example. It is not evidence for an unrestricted complete motive search, but it is a concrete precedent.
Type classes	Hoog+ includes dictionary translation. Lean’s dependent instance constraints extend that problem; they do not introduce it from scratch.
Transport example	For Lean’s natural addition, $n + 0$ reduces to n . Use $0 + n$ versus n to illustrate a genuinely propositional index equality in the usual open-variable setting.
Deterministic results	Sorting after parallel search removes arrival-order presentation differences, not differences in which candidates were discovered.

The bibliographic corrections are supported by the cited primary papers and source code, rather than by the proposal’s explicitly provisional recollection of the literature [21, 30, 26, 20, 19]. The mathematical corrections are proved or explained below. None implies that the overall improvement program is misguided: they make its boundaries more precise and identify techniques worth reusing.

2 R1: search control with coupled goals

2.1 R1.1. Can coupled goals be organized without wholesale copying or sequentialization?

Answer. Yes, in the useful sense of sharing persistent state and exploiting certified independence. No representation removes the underlying combinatorial alternatives when the constraints genuinely couple the goals. The right unit of independence is a *constraint component*, not a goal name.

Aesop is a relevant existing implementation of best-first Lean proof search with dependent subgoals; its handling of shared metavariables is one reason a conventional independent AND/OR tree is insufficient [15]. For Leant2, retain a joint branch state but factor its constraint graph whenever a sound dependency analysis permits it.

A state should contain a persistent metavariable snapshot, open goals, delayed constraints, observations, and an immutable environment identity. For each goal or observation, compute a conservative support: all unassigned expression and universe metavariables reachable through its expression, their types, local declarations and local values, and delayed assignments. Instance-search dependencies belong in that closure as well. Two targets containing no visibly shared metavariable may still be coupled through the type of a local or a universe constraint.

Build a hypergraph whose vertices are unresolved metavariables and whose hyperedges are obligations or constraints. Connected components can be scheduled separately only when the applicable rules are also *effect-confined*: they may write component-local metavariables and fresh component-local variables, but may not silently connect components through a global mutable constraint. A new cross-component constraint merges components. In a persistent representation, merging need not copy all their old terms.

Proposition 2.1 (A sufficient factorization criterion). *Suppose a search state partitions into components $S_1, \dots, S_k$ with disjoint writable metavariable sets. Assume their constraints factor as a conjunction, all shared inputs are immutable, and every local transition and completion respects this partition. Then compatible satisfying substitutions for the components can be combined, and local search order does not change the set of combined solutions.*

Proof. Each component substitution acts on a disjoint domain and does not introduce references to another component’s writable variables. The substitutions therefore commute. Each conjunct depends only on its own substitution and immutable inputs, so it remains satisfied after the other substitutions are applied. Conversely, restricting any global satisfying substitution to the component domains yields satisfying component substitutions. This proves the correspondence. The effect-confinement assumption is essential: initial disjointness alone does not constrain what a later tactic may write. $\square$

Memoization. First memoize *closed, contextual solutions*, not arbitrary open solver states. Abstract a solution over precisely its used local telescope, canonicalize binder identities, record the environment and transparency regime, and recheck its instantiation at the reuse site. A

proof of $P(x)$ is not a proof of $P(y)$ merely because its printed target looks similar. For open solutions, a cache entry must include interface substitutions and residual constraints; keeping only the first returned term can lose the only globally compatible solution.

The general implementation is a persistent search DAG with explicit component interfaces. A conservative first implementation can share only closed lemmas and keep the existing continuation discipline inside each connected component. This offers most of the correctness benefit without first building a full concurrent constraint solver. Measure retained memory as well as save/restore time: a cheap snapshot operation does not imply that retaining a large frontier is cheap.

2.2 R1.2. What admissible heuristic survives shared metavariables?

Answer. Start with $h(s) = 0$. Add only lower bounds proved by a cost-preserving relaxation. The proposed “one unit per open non-leaf goal” is not admissible under the proposal’s own zero-cost exact rule, and coupling creates a second independent problem.

For example, suppose two open obligations can both be closed by assigning one shared metavariable to a local witness. If that assignment costs one, the true remaining cost is one, not two. Even without sharing, an exact-local step assigned cost zero defeats a positive unit lower bound.

Let $h_i(s)$ be a lower bound on the cost of satisfying obligation i when the other obligations are relaxed. Then

$$h_{\max}(s) = \max_i h_i(s)$$

is admissible: any full completion must also solve each relaxed problem. A sum is justified only across independent components whose future costs are charged disjointly. Independence of formulas alone is insufficient if a single charged production or shared subterm serves several components.

A stronger principled construction uses an abstraction α from concrete states to a finite relaxed state space. Require every concrete transition $s \rightarrow s'$ of cost c to have an abstract path from $\alpha(s)$ to $\alpha(s')$ of cost at most c , and require concrete solutions to map to abstract solutions. The shortest abstract completion cost is then a lower bound.

Proposition 2.2 (Abstract-distance heuristic). *Under those simulation assumptions, $h(s) = d_{\text{abs}}(\alpha(s), \text{Solved})$ is admissible.*

Proof. Map any concrete completion path to the concatenation of its simulating abstract paths. The latter reaches an abstract solved state at no larger cost. The shortest abstract path is no more expensive than this one. Taking the infimum over concrete completions proves the claim. $\square$

A type-head hypergraph, a small pattern database of constructor obligations, or a fractional covering relaxation can instantiate this construction, provided the simulation is actually proved for all productions. A fractional covering bound, for example, must permit one action to cover every obligation it can solve by unification. Forgetting those joint effects makes the relaxation an overestimate instead of a lower bound.

Use nonnegative fixed production costs. Give every genuinely generative cycle positive cost, or normalize its zero-cost closure finitely before queue insertion. Otherwise an infinite sequence of zero-cost expansions can starve a finite-cost solution. With an inconsistent heuristic, allow reopening states; alternatively retain a simpler uniform-cost baseline until measurements justify the added complexity.

2.3 R1.3. Does just-in-time learning preserve meaningful ranking?

Answer. It preserves acceptance correctness when it only proposes terms, but it generally changes the candidates found before a deadline. That is a normal consequence of adaptive search, not a kernel issue. Probe supplies a published example of learning a probabilistic grammar from partial successes during enumeration; it does not establish that a finite-time candidate list is invariant under such updates [16].

Separate two numbers:

$$c(p) = \text{fixed advertised ranking cost}, \quad q(s, t) = \text{possibly adaptive expansion priority}.$$

A surviving observation can improve q without rewriting the meaning of c . Record the policy version, observations used for updates, and tie-breaking rule. Freeze learned priorities within logical epochs; updates take effect at a specified event boundary. Rebuild or correctly rekey the queue when priorities change. Quietly leaving stale keys in a heap gives neither the intended adaptive order nor a reliable reproducibility story.

Maintain a fair baseline lane with a fixed grammar and cost order. If an optimality certificate is desired, its frontier lower bound must cover *all* unexplored candidates in the advertised grammar, including those diverted to heuristic lanes. Heuristic search can find an excellent incumbent early; a separate certification frontier establishes when no better candidate remains.

Learning only reorders a finite bounded grammar if proposals actually belong to that grammar. A new helper type outside the bound is an extension, not a permutation. Record the extension explicitly, and never report exhaustion of the original grammar as exhaustion of the enlarged language. This distinction reappears in R9.

2.4 R1.4. Can a wall-clock search be machine-speed independent?

Answer. Not in the strong form proposed, for an arbitrary adaptive search that returns everything it has discovered by a hard deadline. One can obtain deterministic *logical prefixes*, deterministic completed batches, or replay of a recorded run. One cannot generally obtain the same useful discovered set at the same wall time on arbitrarily different machines.

Proposition 2.3 (A wall-clock obstruction). *Consider a deterministic search with a candidate first discovered at logical event j . Suppose event j finishes before deadline D on one machine and after D on another. A policy returning every candidate discovered by D returns different sets on those machines.*

Proof. The first run includes the candidate and the second run has not discovered it. Sorting the results does not add a missing candidate. The contradiction disappears only by changing the return policy, the resource definition, or the deadline guarantee. $\square$

This elementary obstruction does not rule out valuable deterministic interfaces. Use a machine-independent logical work budget for canonical runs and expose a wall-clock cancellation mechanism. A cancellation returns the last completed logical checkpoint, together with an explicit incomplete status. The slower run may return an earlier checkpoint; resuming both to the same checkpoint must give the same candidate stream. Alternatively, reserve conservative fixed work tranches for a known machine class, accepting that an unusually slow machine may not complete the tranche.

For parallel search, assign jobs deterministic identities, allocate logical resources deterministically, and merge completed epochs in a fixed order. A race to the first result followed

by cancellation is not deterministic even if the final sort is stable. Startup calibration improves throughput but does not remove cache effects, load variation, or exceptional expensive reductions.

Universal restart results such as Luby–Sinclair–Zuckerman concern specified models of Las Vegas runtimes. They should not be quoted as an optimality theorem for arbitrary deterministic lanes with correlated goals and reusable state [17]. Leant2’s safer default is a deterministic, resumable portfolio, with measured scheduling improvements evaluated against that baseline.

3 R2: carriers, accumulators, and auxiliary state

3.1 R2.1. Can finite examples determine a minimal carrier type?

Answer. They can determine a minimum *within a precisely bounded synthesis problem*, or the smallest finite-state model consistent with the samples. They do not, without additional assumptions, determine the intended carrier or a unique minimal Lean type. The objective and the bound must include the code that uses the carrier.

A candidate consists of more than K . For a list input, write it as

$$(K, z, s, o), \quad z : K, \quad s : A \rightarrow K \rightarrow K, \quad o : K \rightarrow T,$$

with result $o(\text{foldr } s \ z \ xs)$. Possible notions of minimum include the number of carrier values, the size of a type expression for K , and the description length of (K, z, s, o) . They are not interchangeable. Isomorphic types may have different syntax costs, and a tiny type can require a very complicated step.

A particularly important degeneracy is $K = \text{List } A$, $s = \text{cons}$, $z = []$, and o equal to the entire desired function. This is always a formal “carrier solution” when that function is available, but it has not solved the synthesis problem. Penalize the finishing program and constrain its grammar. Otherwise a carrier search can hide all the difficulty in **finish**.

A terminating bounded algorithm. Fix a finite grammar of type expressions and finitely many term productions, including literal and universe bounds. Bound the total syntax cost of K, z, s, o , not merely the carrier depth. In a fragment with total type checking and evaluation, enumerate candidates in objective order, type-check them, evaluate the finite observations, and return the first success. Exhaustion proves that no candidate in *that bounded grammar* fits the observations. If a checker or evaluator times out, record an unresolved alternative; do not upgrade that run to exhaustive failure. General Lean type inhabitation cannot be replaced by a finite search over type names alone.

For a finite alphabet and a finite carrier of k states, there is a more direct complete decision procedure: enumerate or solve for the initial state, transition tables, and output table. For an alphabet of size a and finite output set of size b , there are at most

$$k \cdot k^{ak} \cdot b^k$$

raw tables before symmetry reduction. Fixing the initial state’s name to zero removes a factor k without losing any nonempty-state model. SAT or finite-domain constraint solving improves the search, but the result is still a sample-consistent state machine, not identification of the unknown target on all inputs.

A sound carrier-exclusion rule

For right folds, the relevant contexts *prepend* a list: $u \mathrel{++} xs$. Let S be a finite set of sampled suffixes and U a finite set of prepended contexts. Whenever observations specify different outputs for $u \mathrel{++} x$ and $u \mathrel{++} y$, put an edge between x and y . Missing observations impose no edge. Call the resulting graph H .

Theorem 3.1 (Finite observation lower bound). *If a deterministic implementation $o \circ \text{foldr } s \ z$ satisfies all these observations, then its reachable states color H properly. Consequently, $|K| \geq \chi(H)$ for finite K ; in particular a clique of size r rules out every carrier with fewer than r values.*

Proof. Associate suffix x with $q(x) = \text{foldr } s \ z \ x$. If $q(x) = q(y)$, prepending the same finite context applies the same sequence of functions $s(a, -)$ to equal states. The resulting states remain equal, and applying o gives equal outputs. Thus endpoints of an edge cannot share a state. This is a proper coloring by elements of K , giving the lower bound. $\square$

This is a necessary condition, not a sufficient transition-consistency test. A coloring may fail to extend to a deterministic transition system. Use it as a cheap certified exclusion before synthesizing the tables or terms.

Here is a better carrier benchmark than reverse. On a one-letter alphabet, request the Boolean output “length is a multiple of three.” Suffix lengths 0, 1, 2, together with suitable prefix lengths in $\{0, 1, 2\}$, are pairwise distinguishable. Thus two-state `Bool` is too small even though the result type is Boolean. Three states suffice, with a cyclic transition and a Boolean finishing test. `Option Bool` has exactly three values and is a candidate from the proposal’s grammar. The accompanying finite-model script checks this example exhaustively.

The count of values *enumerated so far* at a syntactic depth is not a sound substitute for $|K|$. Small code may produce states outside that enumeration, and intermediate fold states need not be among the examples. For infinite carriers such as `Nat`, use a proved abstract-state bound or call the test a heuristic; do not present it as a cardinality refutation.

Why reverse must not be semantically excluded

The classical fold characterization tests whether equal results remain equal after adding a constructor [18]. For reverse there is a direct witness:

$$\text{reverse}(xs) = \text{foldr } (\lambda a \ r. r \mathrel{++} [a]) \ [] \ xs.$$

Proof. At the empty list both sides are empty. For $a :: xs$, the fold returns the induction-hypothesis result followed by $[a]$, which is exactly $\text{reverse}(a :: xs)$. $\square$

The same carrier `List A` is therefore sufficient semantically. With ordinary list append, the displayed implementation performs quadratic list traversal. A function carrier or a left fold can provide a better operational representation. A small grammar lacking append may fail to express the step, but that is a *step-language* obstruction, not a proof that the carrier cannot represent reverse.

3.2 R2.2. What replaces the alphabet for abstract polymorphic elements?

Answer. For an appropriately restricted relationally parametric fragment, shapes and positions are the right starting point. Equality tests between abstract elements are *not* automatically available. The capability to compare elements must come from the signature or a justified semantic assumption.

A list container has shapes $n \in \mathbb{N}$ and positions $\text{Fin}(n)$. A natural transformation from lists to lists is described by an output shape $m(n)$ and a position-selection map

$$\phi_n : \text{Fin}(m(n)) \rightarrow \text{Fin}(n).$$

Its action is to copy the input positions selected by ϕ_n , allowing omission, duplication, and permutation. This representation captures the information that a genuinely parametric program cannot manufacture a new abstract element or branch on its representation. It also forces the empty-input constraints: without another source of an A , the output shape at zero must be zero.

There is directly relevant published work missing from the proposal: Mulleners, Jeuring, and Heeren use container morphisms to automate reasoning about realizability of polymorphic programs from examples and sketches [19]. This is a more precise starting point than an informal appeal to an alphabet of equalities. It does not already provide a complete synthesis engine for arbitrary Lean-dependent signatures.

For a function with a `DecidableEq A` or comparison dictionary, enrich the symbolic domain with the permitted operations and their laws. Equality patterns of finitely many named positions can then be useful; ordered inputs may need order types. A register- or data-automaton interpretation is plausible for such explicitly capability-bounded fragments, but its abstraction must distinguish what the dictionary can observe. Replacing all concrete elements by distinct tokens would lose behaviors that depend on repeated equal values.

Do not apply a free theorem to every kernel-accepted polymorphic Lean term. The allowed constants, axioms, type-class operations, and universe features determine whether a parametricity theorem applies. A safe implementation attaches a proven parametricity certificate to a small syntax fragment or to individual components. Outside that fragment, canonical distinct-element tests are useful probes, not completeness certificates.

Even inside the fragment, testing the canonical list at every length up to n identifies behavior only for those lengths. A program may switch from identity to reverse at length $n + 1$. Thus finite canonical tests support bounded observations, not unrestricted functional equality.

3.3 R2.3. Can fusion be run backwards to invent auxiliary information?

Answer. Yes as a synthesis strategy, and auxiliary-information synthesis already has substantial precedent. The useful target is a coupled system of equations, not a uniquely invertible transformation of a few examples.

AutoLifter formulates lifting problems in which auxiliary programs and combinators are synthesized together, and decomposes these problems to reduce the synthesis burden. Its second-minimum example enriches a summary with additional information; its setting includes a reference program and prescribed algorithmic paradigms [20]. This directly qualifies the proposal's broad statement that synthesis systems do not invent auxiliary state. It does not settle the Lean problem with only a type and sparse arbitrary contracts.

For Leant2, treat state invention as discovering a feature vector q and an algebra:

$$q(\square) = z, \quad q(a :: xs) = s(a, q(xs)), \quad o(q(xs)) = h(xs).$$

The reference h may be a full specification or only sampled observations. A failed attempt to define s exposes two tails that currently have the same summary but require different next summaries or final outputs. Search for an additional feature separating that pair. Pairing adds information; **Option** records an empty/nonempty phase; a function-valued state delays a parameter; an index component tracks a remaining resource.

This leads to a concrete counterexample-guided proposal generator. Maintain a set of required distinctions. Enumerate a low-cost feature from an allowed observation grammar, evaluate how many distinctions it separates, and propose the best enrichments. Then jointly synthesize the update and finishing programs. A failure of one proposed feature is a reason to backtrack, not to conclude that the original carrier class is impossible.

For a polynomial functor

$$F(X) = \sum_{c \in \mathcal{C}} A_c \times X^{r_c},$$

replace the list step by one operation $A_c \times K^{r_c} \rightarrow K$ for each constructor. This supplies a uniform schema for lists and finitely branching trees. It explains why tupling and invariant strengthening recur across datatypes. A bounded enumeration of well-typed carrier expressions and algebra operations is complete for the declared finite grammar. It is not a complete algorithm for discovering arbitrary catamorphisms from finite examples.

Practical first version. Use distinctions to rank a short grammar of result carriers, products, options, and parameter arrows. Keep the carrier-given benchmark track. Measure whether distinction-guided proposals reduce explored terms relative to the same grammar enumerated blindly. This separates a useful invention heuristic from an unsupported claim of optimal learning.

3.4 R2.4. Is there a complete motive enumeration for primitive recursion with parameters?

Answer. There is a simple complete enumeration for an explicitly restricted schema. Avoiding every exponential blowup is a different demand and has no justification merely from the word “primitive.”

Fix finitely many candidate recursor arguments, a finite local telescope, a bounded carrier grammar, and a bound on nested recursion. Enumerate motives of the form

$$M(\vec{i}, x) = \prod_{j \in J} (a_j : S_j(\vec{i}, x, \vec{a}_{<j})). K(\vec{i}, x, \vec{a}),$$

where J is dependency-closed and K comes from the declared grammar. Type-check each complete motive candidate before opening all of its branch searches. The ordinary result motive, one generalized parameter, and a small tuple enrichment should appear early, with explicit costs for extra binders and carrier nodes.

Every primitive-recursive program representable by this schema eventually receives its motive, provided the enumerator is fair and the remaining term grammar is also complete within the chosen bounds. The schema does not include every primitive-recursive program unless the surrounding language and bounds are enlarged accordingly. A recursion-depth bound or outermost-only policy is a useful operational restriction, not a theorem that nested recursion is unnecessary.

For natural numbers, begin with outermost primitive recursion and a small parameter set. Fibonacci can be expressed with the invariant state (F_n, F_{n+1}) and step $(a, b) \mapsto (b, a + b)$; it need not first be found by a two-step recursive-call search. Conversely, primitive recursion with a changing accumulator needs an arrow motive or a generalized parameter. A list of constant result types alone is not complete for that case.

3.5 R2.5. Do generalization and lemma-speculation heuristics transfer?

Answer. They transfer as *proposal mechanisms*, especially when failure traces are represented as equations over partial summaries. They do not transfer as automatically valid pruning rules.

A recurring proof failure can reveal that an induction hypothesis mentions a fixed accumulator while the recursive call uses a changed one. The corresponding synthesis repair is to generalize that accumulator. Two computations traversing the same structure suggest tupling. A failed recursive branch requiring both a result and a structural statistic suggests adding that statistic to the carrier. Anti-unification of several failed obligations can propose a helper specification, but the generalized statement may be false or too strong.

Use a bounded repair language: generalize one dependency-closed local set, add one product component, add one phase tag, or propose one helper with a typed interface. Each repair retains the original contract as the final obligation. Give it a provenance record showing which failed obligations motivated it. Re-run proof checking after the repair; do not import a conjectured helper lemma as an axiom.

The highest-value transfer is a shared *strengthening service* for both programs and proofs. Its inputs are failed transition constraints and failed induction obligations; its outputs are candidate interfaces and a cost. This unifies carriers, generalized motives, and helper contracts without pretending that one generic repair is always successful. An ablation should compare the same repair grammar with and without failure-guided ordering. That is the relevant evidence of transfer.

4 R3: evaluation of programs with holes

4.1 R3.1. Has live bidirectional evaluation reached higher-order examples or dependent types?

Answer. The higher-order part has a concrete positive answer: Smyth’s implementation supports higher-order function examples. Its simplified core presentation restricts these examples, and conflating the core with the implementation understates the result. Smyth also supports interdependent holes and avoids the trace-completeness requirement [21]. Those facts do not establish a ready-made Lean-dependent implementation.

A function example need not be an extensional equality of functions. It can be a finite tree of observations, such as requiring a function-valued result to return w when later applied to v . This is precisely the distinction Leant2 needs for Church encodings: observe the applications actually demanded by the surrounding contract, rather than try to compare function values globally.

The first serious dependent-type obstacle is not merely that a closure captures a proof. It is that the closure’s *type* changes under the captured substitution. A typed hole should have a

contextual declaration

$$?m : (\Gamma \vdash T), \quad ?m[\sigma] : T[\sigma]$$

where σ is a well-typed substitution into Γ . Two occurrences of the same hole can have different instantiated target types. Constraints on their outputs must be compared in the corresponding fibers, with explicit equality evidence when a transport is needed.

A useful first dependent fragment supports Π -types, ordinary indexed constructors, explicit typed substitutions, and finite observations at well-typed arguments. It need not solve arbitrary higher-order unification. An application whose unknown function and unknown argument jointly determine a result should remain a relational constraint when local backward propagation is insufficient. Generating a guessed independent example in that situation can remove valid completions.

There is a second subtlety: an unevaluation rule may produce *sufficient* constraints for one way of satisfying an example without characterizing *all* ways. To use failure of the resulting constraints as a refutation, the alternatives must collectively cover every satisfying completion. Otherwise unevaluation is a proposal or branch-refinement mechanism, not a complete negative test. This distinction should be part of the API's evidence type.

4.2 R3.2. What incremental normalizer can support assignment and roll-back?

Answer. Demand-driven incremental-computation work, including Adapton, provides a sound architectural precedent for dependency tracking and reuse [22]. It is not a drop-in memoized Lean normalizer with all the required backtracking semantics. Implement a small versioned observation cache before a new evaluator.

Cache entries should be keyed by the observation's identity, closure environment, relevant definition environment, evaluator version, and a persistent assignment version. Each entry also stores its *read set*. Invalidation must include variables already read through assignments, not just unassigned blockers.

For example, in branch A let $?m = 0$, so an observation evaluates to true. In branch B let the same search-level hole instead be assigned 1, so the observation is false. In both branches the hole is assigned. A cache keyed by “which holes are still open” cannot distinguish them. Returning to branch A should reuse A's result, not whatever branch B most recently wrote into a global table.

Use immutable assignment nodes or persistent deltas with stable version identities. A cached result is reusable when every assignment and environmental dependency in its recorded read set is unchanged. A conservative implementation invalidates the entire entry on any relevant version change; a more selective implementation validates the read-set fingerprints. Local free-variable values, universe assignments, reducibility options, and local instance evidence can all affect normalization or typing and must not disappear from the key.

Proposition 4.1 (Read-set cache validity). *Let a deterministic evaluator return result r while reading only entries in a recorded set D of an immutable store ρ . If its control flow and primitive operations depend only on those reads, then any store agreeing with ρ on D produces the same result.*

Proof. Replay the evaluation steps. At each step, every read returns the same value, so the same rule and next read are selected. Induction on the finite evaluation trace yields the same result and termination point. $\square$

The assumptions explain why hidden global caches, clocks, and unrecorded type-class lookup invalidate the argument. In the current transaction layer, the work ledger and several caches are `IO.Refs` outside `Meta.SavedState`; the monotone ledger intentionally survives rollback, whereas semantic caches need explicit branch-validity rules [6, 5]. Moving to best-first search should not simply attach the same mutable blocker array to every queued state.

4.3 R3.3. What is the cheapest sound fast evaluator?

Answer. Keep the kernel-based path as the reference, and add a small typed evaluator that either produces checkable evidence or returns unknown. Do not start by reimplementing the whole kernel. Differential testing is necessary engineering evidence, but not a proof of conservative pruning.

There are two useful specifications. A *stable reduction* establishes that every well-typed completion of the holes has the same observed Boolean result. An *abstract evaluator* computes a set of possible results satisfying

$$\{\llbracket O[p] \rrbracket : p \in \text{Comp}(s)\} \subseteq \gamma(\widehat{\text{eval}}(O, s)).$$

This is an *over-approximation* of possible completed behaviors. An empty intersection with the permitted outputs justifies refutation. Under-approximating successful reductions is harmless only when an actual reported reduction is proved stable under every completion; under-approximating possible behaviors is not safe for exclusion.

Lemma 4.2 (Substitution-stable refutation). *Suppose a well-typed open observation reduces to **false** by rules that commute with every admissible completion substitution. Then it is false for every such completion.*

Proof. Apply the completion substitution to every step of the reduction trace. By the commuting assumption each remains a valid step, and the closed Boolean **false** is unchanged. Therefore the completed observation reduces to false as well. $\square$

This is the theorem a fragment evaluator should establish. Start with beta reduction, constructor discrimination, projections, and selected recursor equations on known constructors. For primitives, add a small proof-producing reflective procedure or a trusted fragment theorem. Unsupported terms, exhausted fuel, and typing ambiguity produce unknown. For a performance-first prototype, let the fast evaluator only nominate possible refutations, then confirm each proposed deletion using the trusted path or a checked refutation certificate.

The original `instantiatePartial` explicitly expands delayed assignments for reduction and notes that intermediate metavariables may not retain matching local contexts [5]. This is not a demonstrated counterexample, but it prevents a proof from merely assuming every intermediate term is an ordinary well-typed open Lean term. A verified replacement should carry typed substitution spines, or separately prove the relation between this reduction representation and all well-typed completions.

Lean4Lean is valuable as a mechanized specification and verification effort. Its published work and 2026 project account describe ongoing metatheory and typechecker verification, not a turnkey extraction of every optimized kernel reduction path [23, 24]. Reusing a narrowly proved fragment is realistic. Treating its existence as a completed proof of a new evaluator is not.

First implementation step. Split only contract forms whose decomposition is justified: conjunctions; Boolean conjunction equal to true; and a finite, fully known `List.all` expansion

with a proved equivalence. Preserve the original contract and reconstruction proof. A metamorphic condition $f(x) = f(y)$ must not be turned into unrelated fixed-output examples. Store true/false/unknown separately for each resulting observation and update only invalidated entries.

4.4 R3.4. Can top-down angelic recursion have termination and completeness guarantees?

Answer. Yes for an explicitly finite, fairly explored abstraction with correct witness handling. There is no automatic transfer of an unrestricted termination theorem from Burst to a new top-down Lean engine. Burst’s correctness results have stated assumptions about its angelic-synthesis subprocedure and refinement information [25].

Distinguish three recursive-call treatments. A known observation $f(v) = w$ permits replacing that call by w *conditional on the contract*. A sound over-approximation permits every value that a satisfying completion could return. A guessed value is only one possible witness. Failure under one guessed value does not refute the branch.

Conditional oracle facts require a different statement from unconditional stable reduction:

$$C(p) \implies p(v) = w.$$

When evaluation under these facts yields a contradiction, the sound conclusion is that no completion can satisfy C . It is not that the original observation is false for every arbitrary completion. Keep this dependency in the certificate. If the contract permits several outputs at v , use a set or relation; never select one output and silently regard it as forced.

Recursive calls at equal inputs must receive compatible outputs. In a dependent result family, the compatibility equation lives at the corresponding instantiated result type. Fuel exhaustion means no conclusion. A closed candidate must be evaluated or proved against the original contract without unresolved angelic assumptions.

A simple bounded completeness argument is available when there are finitely many candidate bodies and finite sound oracle domains: enumerate witnesses fairly; after a spurious candidate or witness, remove only a justified finite alternative; never repeat a removed state; and eventually check every candidate exactly. The finite search then terminates and retains every genuine solution. Infinite result domains require symbolic over-approximations or a fair increasing sequence of finite approximations, and termination of the whole increasing search is no longer guaranteed. The useful implementation target is the bounded theorem, not an unqualified global completeness claim.

5 R4: indexed families, motives, and transport

5.1 R4.1. Is maximal-occurrence motive enumeration finite and complete?

Answer. It is finite and exhaustible for a precisely specified finite occurrence grammar. It is not complete for all useful accumulator generalizations merely because it abstracts maximal index occurrences. Generalization must be part of the grammar.

Fix a normalized goal expression, finitely many candidate index expressions, and a finite set of permitted occurrence positions. Enumerating subsets of those positions gives finitely many raw abstractions; type-checking removes ill-formed ones. If the permitted set has k elements, the raw bound is 2^k . This is a termination statement about *syntactic generation*, not a small complexity bound or a complete algorithm for arbitrary higher-order unification.

Maximality is a syntactic convenience. For example, a goal containing $\text{Vec } B \ (n + k)$ may require abstraction of the occurrence of n *inside* the index expression while retaining the parameter k . An occurrence policy that only abstracts the whole maximal expression $n + k$ does not automatically generate that motive. More importantly, abstraction of index occurrences does not introduce a generalized accumulator parameter that did not occur there.

Use a combined grammar with three independent choices: the major premise and its indices; a dependency-closed telescope of generalized locals; and a result carrier expression. Require the resulting motive to type-check before applying the recursor. The type checker is a filter, not a guarantee that every omitted motive is unnecessary.

For vector map, the standard motive is simply

$$M(n, x) = \text{Vec } B \ n.$$

The empty branch builds the empty vector; the constructor branch maps the head with $f : A \rightarrow B$ and combines it with the induction hypothesis at the predecessor length. No arbitrary motive invention is needed here. Indeed, the inspected `structuralRecursion` rejects every inductive family with nonzero `numIndices` *before* calling Lean’s induction API [5]. The first experiment should therefore admit supported indexed locals and call the existing API, rather than first build a general higher-order motive enumerator.

Accumulator examples require more: to prove or synthesize a function whose recursive call changes parameter a , the motive must quantify a rather than freeze it. Dependency closure matters when later locals depend on a . Conversely, reverting every local inflates branch interfaces and search. Rank small dependency-closed generalizations first, with a fair bounded fallback.

5.2 R4.2. Can transports be canonicalized with setoids or heterogeneous equality?

Answer. Neither setoids nor heterogeneous equality provides a general cheap canonical quotient of dependent Lean programs. They can be useful internal relations, but they move proof obligations rather than eliminate them. A typed intermediate language with proof-producing index normalization is a better first design.

A transport is not just a proof argument that may be deleted from every representation. Given $v : F(i)$ and $e : i = j$, the cast has type $F(j)$, whereas v itself has type $F(i)$. Erasing the cast in a displayed definition may make that definition fail to elaborate. Proof irrelevance identifies proofs of the same proposition; it does not make arbitrary index expressions definitionally equal or turn every transport into the identity reduction.

Normalize indices using a fixed terminating rewrite discipline, retaining proofs of each equality. For a small arithmetic index fragment, choose a canonical affine representation and produce a Lean proof relating the original expression to it. Expressions beyond the fragment stay symbolic. Store a cast constraint with source family, source index, target index, and proof obligation. Merge constraints only using checked congruence or a proved canonicalization theorem.

For example, with the usual Lean natural addition, $n + 0$ reduces to n , because addition recurses on its second argument. The source proposal’s `Vec A (n + 0)` example therefore normally requires no propositional transport. The open expression $0 + n$ is the better example: rewriting with the appropriate equality gives the desired index [11]. A normalization ladder should test definitional equality first, then try a bounded proof-producing index procedure.

For ranking, assigning a low or zero *display cost* to proof evidence is reasonable. Keep a positive search-resource charge or a finite transport bound so infinitely many equalities do not create a zero-cost enumeration loop. For a returned program, either print the necessary cast, or print a sugared expression that is re-elaborated and proved equal to the certified term. “Erase it for display” must not mean “publish untypeable Lean.”

Heterogeneous equality is suitable for some intermediate congruence constraints when two terms have different types. Before acceptance, every use must be realized by a well-typed proof term at the intended type. Setoid-based semantics can justify an observational quotient in a specific fragment, but requires all synthesized operations to respect that relation. It is not a substitute for the actual dependent typing judgment.

5.3 R4.3. How do Canonical and sauto manage unfolding?

Answer. The inspected systems do not support a single universal prescription such as “normalize everything up front.” Canonical’s paper translates to its own normal-form representation and discusses unfolding definitions and instances, with possible expression growth. It also gives an indexed-recursion example and a generalized-induction example using a supplied stronger recursor and predicate synthesis [26]. This is a concrete precedent, not an unrestricted complete generalization procedure.

The **sauto** documentation exposes selected and forced unfolding options, as well as heuristic variants. Its project documentation explicitly says that **sauto** and related reconstruction tactics do not themselves perform induction [27, 28]. Thus its useful precedent here is controlled unfolding and dependent elimination, not automatic induction discovery.

For Leant2, use an on-demand transparency ladder. First perform cheap weak-head normalization sufficient to expose a binder, constructor family, or rigid application head. Try unification at the ordinary regime. If failure is blocked by an eligible definition, unfold that definition under a local budget and record the reason. Reserve broad normalization for a closed candidate or a selected expensive proof tier. Distinguish *stuck at this transparency* from *incompatible after a complete decision procedure*.

Memoize normalized forms with the transparency regime, environment version, and assignment dependencies in the key. A failure obtained without unfolding a reducible alias must not permanently exclude a candidate later tried with stronger transparency. Large class dictionaries and recursive definitions are particularly poor targets for indiscriminate full expansion. This ladder is a proposed engineering policy whose thresholds must be measured; it is not claimed as an optimal normalization strategy.

5.4 R4.4. Which Lean motive-computation functions are reusable?

Answer. There are directly reusable functions in the pinned source, but no reason to promise constant-time failure or a version-stable standalone motive oracle. Wrap the actual APIs behind Leant2’s own adapter.

At tag v4.34.0, Lean’s induction source exposes three useful functions: `getMajorTypeIndices`, `mkRecursorAppPrefix`, and `Lean.MVarId.induction`. The prefix builder computes the motive by abstracting the major premise and indices from the target. The induction operation itself reverts indices and the major premise, reintroduces them, and constructs branch subgoals with substitutions. Its index checks include the requirement that indices be suitable variables, distinct in the relevant positions, with permitted dependencies [8].

This has two immediate consequences. First, do not duplicate the index-reversion logic blindly:

the low-level operation already performs much of it. Second, non-variable indices such as $n + 1$ may need the elaborator’s richer generalization path or explicit equations before the low-level API applies. A successful `cases` tactic on an indexed local does not imply that every direct call to this induction API accepts the same input unchanged.

The elaborator’s induction implementation also exposes elimination-application machinery and uses `ElimApp.mkElimApp` and `ElimApp.setMotiveArg` along its tactic paths. Those functions are useful implementation references, but surrounding elaboration state and generalization assumptions matter [9].

The adapter should take a snapshot, normalize only enough to inspect the major premise, compute candidate generalized telescopes, and attempt one motive at a time. Return structured outcomes such as unsupported major type, non-variable index, forbidden elimination from `Prop`, unsolved universe constraint, ill-typed abstraction, or resource exhaustion. Preserve the original Lean diagnostic as detail. Use a bounded resource context and restore all speculative state after failure. Cancellation and heartbeat exhaustion are not ordinary type errors. Finally, pin and compile a small adapter-test suite against the repository’s actual toolchain before calling this a supported Leant2 API.

6 R5: recursion capabilities and realization

6.1 R5.1. Does search-time evaluation agree definitionally with realized recursion?

Answer. Not in general. Propositional agreement is the correct general contract; definitional agreement is a useful special case for particular structural encodings. Well-founded recursion should be evaluated through proved equations or a verified fuelled semantics, rather than by assuming its kernel representation reduces like an ordinary recursive program.

Lean’s reference explicitly demonstrates well-founded definitions whose expected unfolding equation fails by `rfl`, and then proves the equation using `WellFounded.fix_eq`. Generated equation theorems hide the packing of parameters and termination evidence [13]. This directly answers the first part of the question: the capability interpreter cannot claim blanket agreement “up to kernel conversion.”

Let the search body b use a capability

$$\text{rec} : (y : X) \rightarrow R(y, x) \rightarrow Y(y),$$

where R is a fixed well-founded relation for this candidate scheme. A realization pass constructs $P_b : (x : X) \rightarrow Y(x)$ and proves its unfolding equation. A search interpreter $\text{eval}_b(k, x)$ has a fuel parameter and returns either a typed value or unknown. Its essential theorem is

$$\text{eval}_b(k, x) = \text{some}(v) \implies P_b(x) = v. \quad (1)$$

A separate adequate-fuel theorem, when available, establishes that enough fuel produces a value. Soundness (1) does not require a computable universal fuel bound. Fuel exhaustion does not refute P_b .

Proposition 6.1 (A sufficient interpreter–realization bridge). *Suppose every body operation has a proved interpretation theorem, every recursive call carries a proof of decrease, and the realization satisfies the body’s unfolding equation. If the interpreter recursively evaluates only those calls and combines results according to the same body operations, every successful finite evaluation agrees propositionally with the realization.*

Proof. Use well-founded induction on the current argument. Each interpreted recursive call is at a strictly smaller argument, so the induction hypothesis identifies its returned value with the corresponding realization call. Substitute these equalities into the interpretation theorem for the body operations, and then use the realization’s unfolding equation. This gives the equality for the current argument. Equivalently, one may induct on a successful finite evaluation derivation when the interpreter semantics records the recursive calls explicitly. $\square$

For closed examples, several implementations of this contract are reasonable. Rewrite with the generated equation lemmas and discharge resulting decidable equalities. Or reify the admitted body fragment, evaluate it with an ordinary structurally recursive fuel interpreter, and use (1) to reconstruct the observation proof. Compiled execution may cheaply order candidates or nominate a counterexample, but a strict pruning guarantee requires confirmation. A candidate that passes fast tests still needs the original contract proof.

Publishing executable definitions

The compilation issue in E1 is real in the reported probe, but `implemented_by` is not an equality proof. Prefer a generated equation-style definition plus a checked equivalence to the certified logical term. A particularly relevant existing pattern is Lean’s `Nat.recCompiled` and the `@[csimp]` theorem `Nat.rec_eq_recCompiled`: the pinned core source explicitly supplies a compiled implementation and a proved equality [10].

This suggests a small publication adapter rather than a general trusted recursor decompiler. For each admitted recursor scheme, generate the recursive definition, obtain its equation theorems, and prove the bridge once per scheme or once per candidate as appropriate. A compiler simplification theorem or a published equivalent definition can then connect the executable code to the mathematical term. Audit the equivalence proof’s axioms too; they may not already occur in the original program’s axiom closure.

A definition that cannot be compiled should remain a valid logical result with an explicit non-executable status. A definition that compiles but lacks the bridge is not automatically a certified implementation of the original term. Mathematical proof validity and runtime implementation trust are separate entries in the receipt.

6.2 R5.2. Does angelic execution compose with top-down search?

Answer. Yes. The top-down direction is not itself an obstacle; maintaining the right constraints and refinement alternatives is the difficult part. Smyth already supplies a top-down example of recursion with incomplete traces, while Burst’s contribution gives a different, bottom-up angelic construction [21, 25]. Their data structures and completeness assumptions should not be conflated.

In top-down search, an unknown body produces typed hole closures at observed recursive arguments. Store constraints on those closures and propagate only sound consequences. Repeated calls at the same input share an oracle variable, or receive a consistency constraint. If the candidate body is subsequently filled, discharge every oracle fact by the actual recursion semantics. Inconsistent witness assignments are rejected without concluding that every witness for the body is inconsistent.

Compared with a bottom-up tree-automaton method, the top-down engine gains strong contextual typing and natural sketches. It loses some immediate canonical sharing of equal bottom-up behaviors, and it cannot assume that a finite set of partial evaluations is a congruence

for every future context. Search states must include the current environment, substitutions, and oracle constraints. Typed specialization and closed subterm caches recover some sharing without changing these semantics.

The safest first integration uses angelic values only for priority. A later version may use impossibility in a proved over-approximation to prune. Reuse R3’s observation infrastructure and realization theorem; a separate unverified recursive evaluator would duplicate the hardest correctness boundary.

6.3 R5.3. Is there a canonical small basis with measured textbook coverage?

Answer. A useful engineering basis can be specified; no canonical minimal basis or coverage percentage for Leant2 follows from the inspected literature. A single universal recursor can make a basis syntactically tiny while making every example difficult. The meaningful objective is coverage at bounded synthesis cost under a fixed component environment.

I recommend the following ordered scheme family, as a proposal rather than a measured result.

Scheme	Typical purpose and admission condition
Structural, one major argument	List map, length, tree folds, powers on naturals. Start with an inferred or supplied motive and constructor-field recursive capabilities.
Structural with parameters	Accumulator reverse, folds with a changing parameter, and <code>zip</code> by recursion on one list while matching the other. Generalize changing parameters.
Product result	Fibonacci, extrema pairs, and result-plus-statistic computations. A product often avoids a more elaborate call scheme.
Bounded course-of-values	Calls on deeper subterms when a direct tuple encoding is inconvenient. Track the allowed constructor paths explicitly.
Measure or lexicographic order	Euclidean-style recursion, division, and merge variants where no single structural argument decreases on every branch. Each call supplies a checked decrease proof.
Tagged mutual or nested scheme	More specialized cases. Use a generated combined definition and its actual induction/termination interface rather than assume the simple structural API handles every family.

These categories overlap. For example, `zip` does not inherently require a simultaneous-two-argument recursor, and Fibonacci does not inherently require course-of-values. Count a benchmark as covered by every admissible reference encoding, then use the cheapest verified encoding for a scheme-complexity metric. Otherwise the measured “coverage of schemes” is mostly an artifact of which reference program happened to be chosen.

For each task, record the first successful scheme, carrier, required components, termination proof cost, and whether examples include recursive-call inputs. Run a carrier-given or scheme-given oracle track to distinguish interface invention from branch-body search. Report the overlap matrix before deciding which schemes deserve native support. This produces the requested measured coverage; none is invented in this report.

7 R6: contracts beyond finite decidable observations

7.1 R6.1. How should value and proof holes be interleaved without an SMT solver?

Answer. Use a dependency-aware scheduler with proof obligations at stable interfaces. Lean propositions do not require an SMT solver to play the role of refinements, but unrestricted proof search is not a cheap substitute for liquid inference. Restrict the first abstraction and prove its transfer rules.

There are three useful stages. During value construction, propagate cheap, proved abstract facts about shape, length, bounds, or membership. Once the value choices on which a local obligation depends are fixed, attempt the local proof with the branch induction hypotheses. After all branch values and local proofs are available, prove any remaining global property of the assembled program. A failure at one proof tier returns a pending obligation, not a refutation.

The proposal says that proof search about a program with holes is not meaningful before those holes are filled. That is too strong. A lemma may establish a property uniformly in a still-unknown component; a branch invariant may characterize all allowed fillings. What is unsafe is letting a failed speculative proof attempt exclude every future filling without justification. Use metavariables as existential construction choices, and explicit binders or interface assumptions when reasoning universally about all future choices.

A non-SMT analogue of liquid abstraction is a finite family of predicates with Lean-proved implication and transfer rules. For instance, abstract a list result by empty/nonempty status and a symbolic length expression. A component signature can carry a theorem saying how it transforms these facts. Abstract conjunction and implication checks reduce to a finite table or small certified procedures; `simp` and `omega` discharge the remaining arithmetic leaves. Synquid is a relevant precedent for refinement-guided synthesis, but its solver-supported refinement discipline is not already an implementation of arbitrary Lean proposition search [29].

Keep separate queues for unfilled values, stable local proofs, and global proof debt. A task’s readiness is determined by its actual dependencies, not a single global “all values first” rule. Instance goals deserve special treatment because they may determine type variables and runtime operations; they are not merely proof debt. The existing rotation mechanism is a useful fallback, but its coarse classification should not become the whole scheduling semantics [5].

7.2 R6.2. Which properties follow by induction, simplification, and arithmetic?

Answer. An important sufficient class is characterized by a locally checkable invariant preserved by the program algebra. Membership in a selected *certificate grammar* can be recognized. There is no reason to promise a syntactic test recognizing every theorem that an unrestricted induction tactic could eventually prove.

Let a structurally recursive program on $D = \mu F$ have algebra $a : F(K) \rightarrow K$. Let $I(x, r)$ describe the desired relationship between input subtree x and result r . For each constructor c with recursive children $x_1, \dots, x_m$, require a branch certificate

$$\left(\bigwedge_{j=1}^m I(x_j, r_j) \right) \implies I(c(\vec{v}, \vec{x}), a_c(\vec{v}, \vec{r})). \quad (2)$$

Nonrecursive fields $\vec{v}$ and ordinary hypotheses are universally quantified. Base constructors are the $m = 0$ case.

Theorem 7.1 (Algebraic invariant lifting). *If every constructor has a certificate of the form (2), then $\forall x : D, I(x, \text{fold}_a(x))$.*

Proof. Induct structurally on x . For a constructor with children x_j , the induction hypotheses give $I(x_j, \text{fold}_a(x_j))$. Instantiate (2) with these recursive results. The fold’s computation equation identifies the resulting right-hand side with $I(x, \text{fold}_a(x))$. $\square$

This is the exact justification for synthesizing a value and proof together. For a dependent family, replace K and I by the appropriately indexed motive and carry the transport obligations explicitly. For a parameterized accumulator, quantify the parameter in the induction hypothesis whenever recursive calls change it.

A recognizable first proof class allows constructor equations, projections, equality congruence, fixed terminating simplification rules, and arithmetic obligations in a chosen decidable linear fragment. The recognizer checks that each generated branch obligation belongs to this fragment. Then a complete solver for that fragment can discharge it; otherwise the problem goes to the heuristic portfolio rather than being declared false.

For vector or list map, the length invariant reduces in the constructor branch from $|r| = |xs|$ to $1 + |r| = 1 + |xs|$. That is a local arithmetic closure. Reverse involution often needs an append or accumulator lemma. Sorting needs a permutation and order invariant that is stronger than a few closed sortedness examples. Properties involving nested uses such as $f(f(x)) = x$ may require an auxiliary theorem relating the second invocation to the first; the program’s recursion alone does not make the proof local.

The practical recognizer should therefore report one of three things: a local certificate successfully checked, outside the current certificate grammar, or unresolved within its resource budget. It should never use the latter two as logical evidence that the candidate lacks the property.

7.3 R6.3. Which automated-induction facilities are usable in Lean?

Answer. Start with the ordinary induction APIs and Lean’s existing functional-induction support, then add bounded proof portfolios. Do not assume that a general-purpose solver discovers the necessary induction, generalization, and helper lemma automatically.

The pinned v4.34.0 source implements the `fun_induction` tactic through the function `evalFunInduction`. It looks up a function’s generated induction theorem, generalizes the selected targets, and passes the eliminator to the induction machinery. The same source has `fun_cases`. Thus the proposal’s fallback can use an existing function-aligned induction facility once a candidate is published in a suitable equation-style form [9]. Anonymous recursor terms may first need that publication step.

Use `simp`, `omega`, and `grind` as branch solvers with separately measured tiers. Aesop provides customizable proof search and is a useful optional library; custom induction rules may be registered, but that is different from general automatic induction discovery. Canonical supplies a more specialized optional comparison point. LeanHammer-style retrieval and proof search are another optional lane with external dependencies; none should be silently treated as part of a core-only installation [15, 26, 31].

The inspected evidence does not establish a production-ready, general Lean rippling-and-lemma-discovery service with guaranteed latency. That leaves a concrete engineering target: a

bounded induction adapter that tries a short list of major premises and dependency-closed generalizations, simplifies using the candidate’s equations, and invokes the existing branch portfolio. Record which substep consumed time so missing lemmas are not confused with failed induction-variable selection.

For time limits, restore the full tactic/metavariable state after failure and propagate cancellation distinctly. Heartbeats and recursion limits are useful accounting mechanisms, but a wall deadline checked only between search nodes does not preempt every long internal operation. The inspected engine sets the ambient heartbeat limit to zero and uses lane deadlines [2, 6]. Expensive proof-service calls should receive their own resource limits; an external process is the strongest isolation option for an optional heavyweight service.

7.4 R6.4. Is there a principled proof-debt schedule?

Answer. There is a useful elementary principle for a simplified model, but not a universal optimal bandit policy for the actual coupled search. Separate a justified ordering heuristic from the empirical model supplying its probabilities.

Suppose independent proof jobs i and j have fixed positive costs t_i, t_j and success probabilities p_i, p_j , and the process stops at the first success. Trying i before j has expected charged time $t_i + (1 - p_i)t_j$; the reverse order has $t_j + (1 - p_j)t_i$. Therefore i should precede j precisely when

$$\frac{p_i}{t_i} \geq \frac{p_j}{t_j}.$$

This exchange argument extends to an ordered list under the same assumptions. It motivates “probability of useful success per unit cost,” not an optimality claim for all proof debt.

Leant2 violates several assumptions: obligations share information, proofs can be prerequisites for other proofs, a successful local lemma can help many candidates, jobs can be resumed, and a failed tier changes the next tier’s probability. Replacing this with independent arms of an ordinary bandit hides the dependency structure. Even a calibrated probability is conditional on the tactic tier, obligation shape, available lemmas, and previous failures.

A defensible default is deterministic and simple. Reject candidates using cheap confirmed counterexamples first. Attempt stable local obligations before global ones. Give each ready proof task a geometrically increasing resource ladder, record its progress, and reserve a fixed logical share for older pending tasks so they cannot starve. Learn probabilities only as an optional ordering layer over this fair schedule. Treat switching and setup costs explicitly when a service initializes an index or external solver.

The experimental objective should be time to *certified* answer, together with the number and age of unproved survivors. A policy that returns many plausible programs quickly but spends no time proving the right one is not an improvement for this tool. The useful comparison is against fixed tiers and round-robin resumption, not an unsupported assertion that a particular bandit algorithm is optimal.

8 R7: library retrieval and type-guided refinement

8.1 R7.1. How can dependent and universe-polymorphic types be abstracted usefully?

Answer. Use a staged over-approximation that retains rigid constructors, shared variables, and explicit obligation edges. It should deliberately forget some indices, then refine only the

distinctions implicated in failed applications. A graph of result-head names alone is too weak to support the claimed reasoning.

Consider a component of type

$$\forall\alpha, \quad \text{List } \alpha \rightarrow \text{Option } \alpha.$$

The two occurrences of α are the same variable. Replacing them with independent wildcards introduces spurious paths such as a list of naturals becoming an option of Booleans. That may be an acceptable initial over-approximation if failures refine it, but it is wasteful. Preserving repeated-variable relationships is cheap and removes an entire class of spurious paths.

For an indexed component, retain the type constructor and a compact index pattern: a variable, zero, successor, a small affine expression, or unknown. For universes, preserve obvious equalities and inequalities and the distinction between unrestricted elimination and a recursor restricted to **Prop**. Do not reject a type merely because the abstraction cannot resolve its universe constraints. It must over-approximate the actual legal instantiations.

Components with multiple premises form *hyperedges*, not ordinary edges: all input obligations must be supplied in a compatible context. Type-class dictionaries and proof prerequisites are additional inputs. A single path through type heads does not account for argument multiplicity, shared substitutions, or linear-style restrictions that Leant2 may impose on a consumed Church eliminator.

A proposed pipeline is: cheap rigid-head retrieval; compatible argument-pattern filtering; relaxed hypergraph reachability; exact dependent application; and a small abstraction refinement after a failed application. The refinement should be reusable, such as a missing index equality or a shared-parameter constraint. Merely rejecting the one concrete path without strengthening the abstraction is not the full type-guided abstraction-refinement loop.

Proposition 8.1 (Safe abstract reachability exclusion). *Suppose every concrete well-typed grammar derivation maps to an abstract derivation reaching the abstraction of its result. If the abstract search exhaustively establishes that the requested abstract result is unreachable, then no concrete grammar derivation constructs that result.*

Proof. A concrete derivation, if one existed, would map to an abstract derivation by the simulation assumption, contradicting abstract unreachability. $\square$

This is the correct theorem for a retrieval-based exclusion. An incomplete abstract search or an under-approximating index cannot support the premise. In ordinary interactive operation, retrieval should usually order a limited provider pool rather than claim a global impossibility result. Widen the pool fairly or report the pool boundary in the receipt.

A discrimination-tree index is still worthwhile, but “cost proportional only to query size” is not a general complexity guarantee. Wildcards, polymorphic providers, and a genuinely large result set can require inspecting many candidates. Measure retrieved-set size, exact-unification attempts, and total retrieval cost under growing libraries, not just the tree lookup itself.

8.2 R7.2. Has type-guided refinement handled type classes already?

Answer. Yes. Hoogle+ explicitly translates Haskell-style class constraints to dictionary types and dictionary-producing components; instances become ways to construct the required dictionaries. This is described in its implementation section, including examples for equality dictionaries [30]. It is a direct published answer to the question, though Lean’s dependent and higher-kinded cases require additional work.

Use the same basic representation: a constrained component takes a dictionary as an explicit input in the search model, even if Lean syntax later inserts it implicitly. The difference in Lean is that class parameters may include values, multiple related types, output parameters, universe variables, and local instances. Dictionary resolution can assign metavariables, so it belongs to the shared constraint state.

A cache keyed by only a class name and one type is insufficient. Key it by the fully instantiated class expression, the relevant local-instance environment, transparency and universe dependencies, and assignment version. An unresolved output parameter is part of the interface; a successful dictionary may determine it. The current search already lets instance goals help determine other holes, which should be preserved when retrieval becomes more abstract [5].

Separate three outcomes: a dictionary found with a checked term, a certified incompatible obligation in a restricted complete class fragment, and ordinary resolution failure or timeout. Only the second is a general exclusion. In most real cases failure means “not resolved by this procedure under these settings.” A later local instance, different instantiation, or larger budget can change it.

Do not carry HoogLe+’s evaluation numbers over to Lean’s whole library. The relevant question is whether explicit dictionary edges and a refined dependent index reduce exact application attempts at equal recall. A benchmark should include ordinary classes, local instance overrides, multi-parameter classes, and output-parameter cases that force nonlocal assignments.

8.3 R7.3. Can Lean’s selection interface accept data goals and examples?

Answer. At the interface level, data goals are admissible; examples can be sent as optional hints, but there is no guarantee that a registered selector interprets them. The distinction can be established directly from the pinned source rather than guessed from the word “premise.”

In Lean 4.34.0, `src/Lean/LibrarySuggestions/Basic.lean` defines the central type as

```
abbrev Selector := MVarId -> Config -> MetaM (Array Suggestion)
```

The configuration has a caller tag, a maximum suggestion count, a filter, and `hint : Option String`. The source explicitly says that the hint is arbitrary and may be ignored, and that core Lean provides the interface but no default suggestion engine [12]. The interface type does not require the metavariable’s target to be a proposition. A specific downstream engine may nevertheless be trained on proof goals, filter out useful definitions, or discard observation features.

A first adapter can use `caller := some "leant2.data"`, an appropriate filter for safe component declarations, and a structured textual summary of examples in `hint`. This is an integration experiment, not an assertion that an existing model understands the summary. For reliable example-aware retrieval, define a typed Leant2 request with the target, local context, carrier role, allowed components, and observation features, and explicitly implement the translation for each backend.

Maintain separate data-component and proof-lemma retrieval scores. A lemma useful for proving length preservation need not be a good component for computing a list, while a computational definition’s body and argument types may matter more than its theorem usage history. The current LeanHammer work is evidence for useful neural selection in dependent proof contexts, not a measured result on carrier or data-component invention [31].

For training and evaluation, erase selected definitions from a frozen core, standard-library, and curated user-code environment. Keep only legitimate dependencies available to the synthesizer.

Remove the original constant, aliases, generated equation lemmas revealing its implementation, and accidental specification shortcuts. Split by dependency-connected families, not random near-duplicate declarations. Use held-out types and algorithm families as an additional stress test. A corpus with the answer still available under another name primarily measures retrieval leakage.

Retain a no-learning baseline and a provider-oracle track. Retrieval quality should be reported as useful-component recall at k , exact application success, and end-to-end certified synthesis rate. A proof-retrieval recall score alone does not answer the user’s data-synthesis question.

9 R8: meaningful results and equivalence

9.1 R8.1. What ranking objective is defensible?

Answer. A declared, task-conditioned preference model is defensible; a universal numerical notion of “the intended program” is not. Use correctness as an admission condition and expose the main preference dimensions, even if the interface keeps the default policy automatic.

A Bayesian or minimum-description-length interpretation is useful. For a noiseless specification D , every consistent candidate has the same likelihood under the hard consistency model. Ranking then comes from a prior:

$$-\log P(p \mid T, D, E) = -\log P(p \mid T, E) + \text{constant}, \quad p \models D.$$

Thus “shortest” is a particular prior, not a consequence of the examples. A useful description length may include the program, invented helper definitions, carrier type, and the cost of invoking rare components. Arbitrarily making one library constant free can reverse the ranking, so the grammar and its version belong in the receipt.

Use hard profile constraints first, then a modest Pareto frontier over implementation size, proof/axiom requirements, and a declared operational or readability proxy. If a single default is needed, define a stable lexicographic or scalar key and state it precisely. The smallest proof need not accompany the simplest program, and the smallest program may be asymptotically slower. These choices are preferences, not logical facts.

Do not make input usage an absolute criterion. Constant functions, projections, and lawful interfaces with redundant parameters legitimately ignore inputs. A syntactic occurrence can also be dead code rather than useful dependence. Treat unused-input count as one low-confidence feature, conditioned on the signature and contract, rather than as evidence of an incorrect or meaningless candidate. A contract saying $f(0) = 0$ cannot identify the intended behavior at other inputs; the tool should preserve that ambiguity instead of presenting its prior as a proof of intent.

The inspected engine currently sorts by unused outer binders, eliminator occurrences, and expression size, and deduplicates candidates by pretty-printed text [2]. These are clear places to separate semantic identity, display preference, and discovery order. A stable final tie-breaker should use a canonical serialized expression and dependency identity, not local timestamps or a pretty-printer’s options.

An anytime stop criterion needs the same cost everywhere. If p^* is an incumbent and $L(s)$ is a proved lower bound on the final ranking cost of every completion of frontier state s , then

$$\min_{s \in \text{frontier}} L(s) \geq c(p^*)$$

certifies that no strictly cheaper candidate remains, provided the frontier covers all remaining alternatives. It does not certify a richer top- k diversity objective, and it does not apply when

the search queue is ranked by an unrelated learned probability. For k distinct semantic classes, the frontier bound and duplicate policy must be defined over those classes as well.

9.2 R8.2. What inexpensive equivalence is safe for dependent programs?

Answer. Use a hierarchy, and make the level explicit. Canonical syntax, definitional equality, proved extensional equality, and agreement on finitely many observations are different relations with different permissible uses.

Canonical alpha-renaming and safe local reductions provide cheap exact keys. For two closed terms at the same target type, a successful definitional-equality check provides stronger evidence, but a resource-limited failure means unknown. A proved equality may justify merging terms extensionally even when conversion does not. For function types, that may require a function-extensionality proof; its axioms must respect the chosen profile.

A fingerprint of outputs on a finite test set is only an observational cluster. To see why, fix tested natural inputs $0, \dots, b$ and compare

$$f(n) = n, \quad g(n) = \begin{cases} n & n \leq b, \\ n + 1 & n > b. \end{cases}$$

They have identical fingerprints and differ at $b + 1$. The same phenomenon occurs for lists, including shape-parametric functions that change their positional action after a length threshold. Finite observations do not prove equivalence.

For *closed accepted results*, clustering is useful display compression: show one representative, retain alternatives, and record the tests that define the cluster. New probes can split it later. For *partial search states*, deleting every but one fingerprint representative is sound only if the relation is a congruence for every remaining grammar context and the associated proof/interface constraints. That requires a fragment-specific theorem; it is not implied by matching current examples.

Dependent results add a typing requirement. Values at $F(i)$ and $F(j)$ cannot be compared by an ordinary equality until their indices have been related and a transport chosen. Include index and family identity in fingerprints, or normalize them with certified equality evidence. For a dependent function $(x : X) \rightarrow F(x)$, compare outputs pointwise in each common input fiber. Comparing arbitrary serialized outputs after erasing their indices is not a semantic equivalence test.

Finally, distinguish extensional equality from implementation equivalence under a cost model. The quadratic reverse fold and a linear accumulator reverse can be extensionally equal while materially different as implementations. A user asking for useful synthesized code may prefer retaining both until the ranking policy accounts for that difference.

10 R9: model-guided proposals with independent certification

10.1 R9.1. What accuracy do current models achieve on carrier and helper proposals?

Answer. The inspected sources do not establish an accuracy figure for the exact Leant2 task: proposing a useful typed carrier or helper interface, without supplying the whole implementation, under the proposal's component restrictions. General proof, code-generation,

or premise-selection scores cannot answer that question. A numerical answer would require an experiment; none was run here.

Define the event before measuring it. A carrier proposal is *well formed* if it elaborates in the allowed grammar. It is *expressive* for a task if some allowed update and finishing program can meet the contract. It is *useful at budget B* if the fixed downstream synthesizer actually finds and certifies such a program within that budget. Those probabilities can differ dramatically. Counting a plausible-looking product type as a successful answer measures the first, easiest event at best.

Run four paired conditions with the same downstream engine: blind carrier enumeration; distinction-guided symbolic proposals; a fixed recorded model proposal stream; and an oracle carrier supplied by a verified reference solution. Use identical component environments and either equal total time or clearly separated synthesis-only and end-to-end time. Include model initialization, inference, malformed outputs, and rejected proposals in end-to-end cost.

Report top-1 and top- k well-formedness, certified end-to-end success, useful-proposal latency, and the residual gap to the oracle condition. Stratify by first-order state, function carriers, indexed motives, and helper lemmas. Use disjoint algorithm families for held-out testing, keep prompts and model versions fixed, and publish every proposal rather than only successful ones. Confidence intervals should be computed on independent task units or with family-level resampling when tasks share structure; many nearly identical list variants are not many independent observations.

A successful model-assisted run proves that particular proposal useful under the recorded configuration. It does not prove that learning is necessary or that the carrier is minimal. The oracle condition is especially diagnostic: a large remaining failure rate means branch search or proof search, not carrier invention, is still the main bottleneck.

10.2 R9.2. What makes model-assisted search deterministic and auditable?

Answer. Freeze the proposal stream for search replay, restrict the proposal language, and validate the final term independently. Temperature zero is not, by itself, a cross-platform reproducibility guarantee. Search replay should not require regenerating identical model outputs.

Accept model output only as a small typed proposal language: an allowed carrier expression, a helper signature, a component ordering, or a recursion-scheme choice. Do not execute arbitrary suggested tactics, imports, attributes, commands, or foreign code. Such code can affect the elaboration environment and the meaning of later statements, not just propose a harmless term. Lean’s proof-validation guidance makes the distinction between a valid proof and the intended statement explicit [14].

A receipt should record the complete proposal bytes, their canonical parsed form, all rejection reasons, the model and tokenizer identifiers, prompt/template version, decoding settings, and a hash of the allowed environment and grammar. To reproduce the mathematical result, store the closed program and proof. To reproduce the search, replay the accepted/rejected proposal stream and logical scheduling events. Reproducing the model’s own numerical inference additionally requires its weights, backend, and numerical configuration and may be a stronger, separate service.

Proposal scope. An ordering suggestion within G_B changes only discovery order. A carrier expression not already represented in G_B changes the search space. The latter may be valuable, but label it an extension $G_B \subseteq G'_B$ and record which results used it. A model failure or absence of a proposal cannot justify a negative result about either grammar. Maintain the

symbolic baseline lane.

Four audit levels. A proof receipt supports independent checking of the frozen statement and axiom policy. A realization receipt supports checking the executable program’s equivalence to the logical term. A search receipt supports replay of the explored bounded grammar and scheduling decisions. A model receipt supports attribution of proposals to the recorded inference configuration. Keeping these levels separate prevents “kernel checked” from being misread as “search exhaustive,” “compiled behavior independently proved,” or “model deterministic.”

The model remains outside the mathematical trusted base when proposals are fully validated and cannot alter the checker, environment, or original statement. It can still damage usefulness by wasting time, biasing ranking, or hiding alternatives. Those are addressed by accounting, replay, grammar tracking, and fair fallback, not by proof checking alone.

11 An integrated implementation plan

The preceding answers favor a small set of shared services over nine largely independent research features. The central objects are a frozen query, a typed search state, an observation graph, a proposal stream, and a certificate. Carrier invention, recursion, and proof generalization become different producers of proposals. Retrieval and statistical models supply alternatives rather than bypassing these objects. Evaluation supplies either a justified refutation, a justified fact, or an explicitly incomplete signal.

11.1 Keep the semantic interfaces narrow

The following is architectural pseudocode, not code checked against Lean’s metaprogramming API. In particular, the evidence fields require an implementation of the corresponding certificate checker or a clearly identified trusted invariant.

```
FrozenQuery = {
  environmentIdentity, target, contract, axiomPolicy,
  grammarIdentity, rankingPolicy, resourcePolicy
}
SearchState = {
  persistentMetaState, typedGoals, constraints,
  observationState, logicalCost, tracePosition
}
ObservationResult =
  Established(evidence)
  | Refuted(evidence, scope)
  | Residual(typedValue, readSet)
  | Unknown(reason)
Proposal = {
  decisionPoint, syntaxOrExpression, provenance,
  grammarMembership, deterministicPriority
}
RealizedCandidate = {
  logicalProgram, optionalExecutable,
  optionalEquivalenceProof, contractProof
}
```

The `scope` of refutation matters. A reduction independent of holes may refute every completion. An oracle-based argument may refute only completions satisfying the original contract. An exhaustive small grammar test may exclude only a declared subgrammar. A timeout excludes

nothing. These are different constructors or checked tags, not comments attached to an undifferentiated Boolean.

There should be one path from a realized candidate to acceptance. That path reconstructs the required proposition from the frozen query, checks the program and proof, audits axioms, and records the environment. A separate path may publish an executable after its connection to the logical program is established. Neither an evaluator result nor a retrieved declaration name is a certificate by itself. A model-produced tactic script can be attempted in an isolated elaboration context, but its claims are still untrusted until checked.

11.2 Implement in dependency order

Stage A: preserve and expose the baseline. Add a question-independent regression harness, structured outcomes, source-pinned receipts, and explicit separation of logical certification from successful compilation. Stabilize candidate identifiers and tie-breaking. Keep the present depth-first engine as a reference implementation and a fallback. The existing continuation discipline is valuable: it provides a known comparison when the state representation changes [5, 6, 2].

The first acceptance tests are qualitative. Every previously accepted program must still check against its original target and contract. An intentionally unrelated proof must be rejected by the hardened public gate. A program that cannot be compiled must not produce a misleading runtime-success message. An interrupted proof attempt must not be reported as a proof of impossibility. For strict profiles, an equality bridge using an unapproved axiom must be rejected even when its executable counterpart works.

Stage B: remove avoidable blind spots before inventing a new search engine. Experiment with carefully bounded natural-number and indexed-family recursion, proof-side library lookup, and richer observation decomposition. In particular, the inspected `structuralRecursion` guard rejects indexed families before calling Lean’s induction machinery; the native machinery already computes ordinary indexed motives [5, 8]. Test `Vec.map` and simple `Nat.rec` examples first. This does not imply that removing the guard globally is safe for performance or that all indexed datatypes will work.

Build proof-producing observation adapters for conjunction, Boolean conjunction, and finite list checks. Keep each observation’s original proposition, its closed inputs, its dependency information, and a proof explaining the decomposition. Add a versioned cache initially for cases whose expressions and dependency closure are easy to characterize. Compare every cached result with the uncached path in a diagnostic run.

Stage C: introduce first-class search states without changing policy. Re-express the current continuation as a deterministic state machine with an explicit stack. Demonstrate equivalent event traces under the same logical budget before replacing the stack by another frontier. Separate backtrackable state from intentionally monotone accounting. The current `IO.Ref` ledger is outside `Meta.SavedState`; a future heap of saved metavariable states must not assume those references roll back automatically [6].

Then add a uniform-cost frontier with a zero heuristic, a proved relaxation, or a clearly heuristic scheduling policy. Preserve a fixed candidate-ranking objective separately. Benchmark frontier memory as well as node count: inexpensive persistent snapshots do not imply that retaining a large frontier has negligible cost. Add component factorization only after the dependency closure and effect-confinement conditions can be tested.

Stage D: share a bounded strengthening grammar. Implement result carriers, one parameter arrow, small products, and `Option` states as one proposal source. Add a finishing

function as an explicit synthesis obligation. Use sound incompatibility witnesses only for a genuine finite-state bound; otherwise use distinctions to rank proposals. Feed the same proposal mechanism into induction generalization and helper interfaces.

At this stage the carrier-given track is essential. A task that remains difficult even with its reference carrier is primarily a body-search or evaluator problem. A task solved quickly with a carrier but slowly without one measures invention. Combining these scores would conceal the very distinction the architecture is supposed to improve.

Stage E: expand semantics only where a certificate path is ready. Add typed hole closures, limited unevaluation, recursive-call capabilities, and program-proof co-synthesis one admitted fragment at a time. Each new fragment needs a simulation argument, explicit unsupported cases, and differential tests. Well-founded recursion should enter through a realization interface with equation lemmas, not through an assumption that the search interpreter and kernel are definitionally identical.

Stage F: add scalable and learned proposal sources. Introduce refined library abstractions, then optional model guidance. The default core-only path must remain useful and reproducible without downloading a model or connecting to a service. Record exactly which alternatives came from optional sources, their grammar membership, and the deterministic order in which they were admitted. Separate a model’s effect on proposal quality from its effect on total interactive latency.

These stages are dependency gates, not calendar estimates. Moving to a later stage is justified by a closed test obligation and a measured bottleneck, not by completing a predetermined number of weeks.

11.3 Concrete source integration points

Inspected component	Suggested first change
<code>Leant2/Core/Types.lean</code>	Add an uncertified-survivor outcome, query-linked certificates, and explicit distinctions between timeout and exhausted checked grammar.
<code>Leant2/Accept/Gate.lean</code>	Derive expected types from the frozen query; distinguish logical checking from executable publication and audit any bridge proof.
<code>Leant2/Native/Transaction.lean</code>	Classify every mutable field as branch-local, query-monotone, or externally observed; preserve deterministic trace accounting across rollback.
<code>Leant2/Search/Core.lean</code>	Start with bounded recursion experiments, observation adapters, and dependency-aware cache keys; only then externalize the frontier.
<code>Leant2/Engine.lean</code>	Introduce stable candidate identities, fixed ranking policy, logical epochs, and receipts that state the extent of exploration.
Lean induction adapter	Wrap <code>MVarId.induction</code> and selected elaboration services with snapshot restoration and structured failure reasons.
Library adapter	Use rigid-head lookup and <code>LibrarySuggestions.Selector</code> ; treat hints as optional features, not a promise of example-aware retrieval.

The table names integration locations in inspected source, not patches applied by this report. Public APIs should hide Lean-version-sensitive details behind a small compatibility layer. The native induction and library-suggestion source inspected for this purpose is pinned to 4.34.0 [8, 12].

11.4 Experiments that can falsify the recommendations

Use three distinct task families: the repository’s existing compatibility suites, the failing stretch and edge cases, and a held-out collection of small Lean definitions with their appropriate contracts. Deduplicate equivalent tasks and separate training from testing by definition family or module. Removing a target body while leaving an equivalent wrapper, its generated equation theorem, or a directly usable reference implementation in the provider set is not a meaningful synthesis test.

For each task, retain the complete query, imports, allowed providers, recursion and carrier grammar, observations, reference implementation, and success criterion. Report time to a *certified* result separately from time to an example-consistent survivor. Record the rank of the intended behavior where the contract underdetermines it; matching the spelling of one reference term is not necessary.

Hypothesis	Controlled comparison	Evidence against it
Per-observation caching reduces work	Same candidate trace, cached versus uncached evaluation	Cache bookkeeping outweighs avoided reductions, or branch replay disagrees.
Distinctions help carrier invention	Same finite carrier grammar and term budget, different ordering	No improvement in certified solves or explored states; significant baseline regressions.
Joint-state search improves control	Explicit DFS versus best-first, same admissible grammar	Frontier memory or expensive state operations erase the gain.
Local proof obligations help certification	Program-first versus subtype-/invariant synthesis	Proof branching dominates, or strengthened invariants exclude intended programs.
Refined retrieval improves scaling	Same task and concrete checker, head-only versus refined index	Index construction and false positives dominate; recall falls without an honest grammar restriction.
Model proposals justify their latency	Enumeration, recorded proposals, and live inference as separate runs	Proposals rarely add useful carriers, or end-to-end certification is worse.

Report certified solved counts at several wall-clock limits, but also compare runs at a fixed logical-work limit. Include peak resident memory, frontier size, evaluator invocations, invalidated cache entries, failed instance searches, proof attempts, and reason-coded timeouts. Cold-start and warm-session measurements should be separate, especially for library loading and optional inference. Use repeated runs and disclose hardware and toolchain; no universal improvement factor follows from one query profile.

For correctness, add adversarial tests: two goals sharing only a universe constraint; a delayed assignment that changes an observation; two branch-local instances for the same class head; a timeout during proof reconstruction; a proposed cast across propositionally but not definitionally equal indices; and two candidates equal on all current tests but different on the

next generated input. The six finite checks supplied with this report illustrate some of these distinctions, but do not replace such Lean-level regressions.

11.5 What should count as a successful next release?

A defensible next milestone is not “general dependent synthesis solved.” It is a source-pinned engine that preserves the existing compatibility behavior, compiles a documented family of certified recursive programs, handles a measured subset of the current edge cases, and reports its limitations precisely. Receipts should say whether the result is merely the best found, optimal for a completed cost layer, or obtained after fully exhausting a declared finite grammar.

The long-term research direction is still ambitious: infer useful auxiliary state and dependent invariants while retaining a simple interactive experience. The immediate strategy is to make each new source of intelligence replaceable and each correctness claim local. That permits aggressive heuristics, including statistical ones, without confusing experimental search quality with verified logical evidence.

A Coverage index: all 31 expert questions

Question labels below preserve the proposal’s R-group and the order within its “Questions for experts” list. Descriptions are paraphrases, not quotations. R8 and R9 share the source file `12-ranking.tex`. Every row points to a substantive answer, not merely a bibliographic pointer.

Question	Subject of the answer	Page
R1.1	Coupled goals, independence, and memoization	5
R1.2	Admissible cost-to-completion heuristics	6
R1.3	Just-in-time learning and ranking stability	7
R1.4	Wall-clock budgets and machine independence	7
R2.1	Constructive carrier inference and bounded minimality	8
R2.2	Polymorphic alphabets, shapes, and positions	10
R2.3	Backward fusion and auxiliary-state invention	10
R2.4	Motive enumeration for primitive recursion	11
R2.5	Generalization and lemma-speculation heuristics	12
R3.1	Higher-order and dependent live evaluation	12
R3.2	Incremental normalization and backtracking caches	13
R3.3	A sound, fast evaluator and Lean4Lean	14
R3.4	Top-down angelic recursion and completeness	15
R4.1	Bounded motive classes and generalization	15
R4.2	Canonical treatment of propositional transport	16
R4.3	Definitional unfolding in existing tools	17
R4.4	Lean’s callable motive-computation services	17
R5.1	Interpreter agreement with realized recursion	18
R5.2	Angelic execution in top-down synthesis	19
R5.3	A practical, measurable recursion-scheme basis	20
R6.1	Interleaving program holes and proof holes	21
R6.2	Recognizable classes of induction-friendly contracts	21
R6.3	Automated induction and Lean integration	22
R6.4	Scheduling proof debt	23
R7.1	Dependent and universe-polymorphic type abstraction	23
R7.2	Type classes in abstraction refinement	24
R7.3	Data-goal retrieval and training corpora	25
R8.1	A formal ranking objective and user preference	26
R8.2	Candidate equivalence in dependent types	27
R9.1	Model accuracy and an appropriate evaluation study	27
R9.2	Deterministic and auditable model contributions	28

B Executable checks and the limits of the evidence

The archive contains `validate_models.py` and its generated `model_results.json`. The script uses the Python standard library and was executed successfully during preparation of this report. It checks six small, explicitly defined models. These checks are useful regression examples for the proposed algorithms; they are *not* Lean proofs, tests of Leant2, or performance benchmarks.

Check	Actual result and interpretation
Reverse as a list fold	All 1,093 lists over a three-symbol alphabet, of lengths 0–6, agree with reversal when evaluated using the step $r \mapsto r++[a]$. The induction proof in R2.1 covers arbitrary finite lists; enumeration alone would not.
Finite carrier inference	Exhaustive unary transition/output tables with a fixed initial state: 2 one-state tables, 16 two-state tables, and 216 three-state tables. On the length-0–6 observations for divisibility of length by 3, there are respectively 0, 0, and 2 solutions. The first witness cycles through three states.
Carrier incompatibility	Suffix lengths 0, 1, and 2 form a three-vertex incompatibility clique. Prefix lengths 0, 0, and 1 respectively witness the three edges. Thus any deterministic fold satisfying those observations needs at least three reachable states.
Coupled-goal cost	One abstract transition of cost 1 closes two obligations. Their naive per-goal sum is 2; a maximum of the individual lower bounds is 1. A separate zero-cost exact move also defeats a positive open-goal count.
Rollback-aware cache	Branches A, B, and A with an irrelevant change produce true, false, and true. A dependency-version key needs two evaluations and permits one cache hit. Merely observing that no holes remain unassigned does not distinguish A from B.
Finite observation classes	The functions $f(n) = n$ and $g(n) = n$ for $n \leq 12$, $g(n) = n + 1$ otherwise, agree on 0–12 and differ at 13, returning 13 and 14.
Approximation direction	Concrete possible outputs $\{0, 1\}$ intersect the desired output $\{1\}$. The under-approximation $\{0\}$ does not. Pruning from that under-approximation would therefore discard a possible solution.

The two carrier rows arise from one model family; hence there are six families, not seven independently developed experiments. State labels are not quotiented when counting the tables or solutions. Fixing the initial state to 0 is harmless for the existence question because an automaton’s states can be renamed.

B.1 Reproduction

From the archive’s source directory, run:

```
python3 validate_models.py
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The first command checks its assertions and regenerates the JSON results. The remaining commands build the article and settle the table of contents, references, and page links. The bibliography is included directly in `references.tex`; BibTeX and internet access are not required to compile the article. A standard TeX Live installation with the packages listed in the preamble is sufficient.

B.2 What was not established

No Lean executable was available in the execution environment used for this report. Consequently, the proposed interfaces and code-shaped examples were not compiled, the mathematical arguments were not formalized in Lean, and the repository’s acceptance and stretch suites were not rerun. Source inspection establishes what the inspected code does, not that every suggested change will compile or improve performance.

The report does not claim that a proposed evaluator is verified, that the suggested grammar is optimal, that a current model reaches a particular accuracy, or that the complete design is already implemented. Published systems supply precedents with their own assumptions and experimental settings. Transferring them to Leant2 requires the staged validation described in Section 11.

The constructive mathematical results here are deliberately conditional: factorization requires effect confinement; the cache lemma requires a complete dependency set; carrier exclusion requires genuine finite-state bounds; bounded completeness requires a fixed finite grammar and justified decisions; and recursive realization requires checked equations and termination evidence. These conditions are part of the result, not optional implementation details.

References

- [1] V. Reshetnikov and Leant2 contributors. *Further-improvements proposal for Leant2*. Repository proposal, September 18, 2026; inspected revision 784664ff34e819422510a4d470ab07fe48bb736b. Especially sections/05-search.tex through sections/12-ranking.tex. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements>.
- [2] Leant2 contributors. *Engine.lean*: query configuration, lanes, acceptance calls, and ranking. Inspected at the repository revision in [1]. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Engine.lean>.
- [3] Leant2 contributors. *Accept/Gate.lean*: synchronous checking and axiom audit. Inspected at the repository revision in [1]. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Accept/Gate.lean>.
- [4] Leant2 contributors. *Core/Types.lean*: profiles, ledger, accepted candidates, and outcomes. Inspected at the repository revision in [1]. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Core/Types.lean>.
- [5] Leant2 contributors. *Search/Core.lean*: providers, partial instantiation, residual evaluation, recursion, and construction search. Inspected at the repository revision in [1], especially lines 1–280 and 410–710. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Search/Core.lean>.
- [6] Leant2 contributors. *Native/Transaction.lean*: saved-state attempts and external work accounting. Inspected at the repository revision in [1]. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Native/Transaction.lean>.
- [7] Lean project. *The Lean Language Reference: Introduction*. Rolling reference consulted September 21, 2026; displayed version 4.35.0-rc2. <https://lean-lang.org/doc/reference/latest/Introduction/>.
- [8] Lean contributors. *Lean/Meta/Tactic/Induction.lean*. Source at tag v4.34.0; in particular getMajorTypeIndices, mkRecursorAppPrefix, and MVarId.induction. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Induction.lean>.
- [9] Lean contributors. *Lean/Elab/Tactic/Induction.lean*. Source at tag v4.34.0; elimination elaboration and evalFunInduction. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Elab/Tactic/Induction.lean>.
- [10] Lean contributors. *Init/Data/Nat/Basic.lean*. Source at tag v4.34.0; Nat.recCompiled and the compiler simplification theorem Nat.rec_eq_recCompiled. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Init/Data/Nat/Basic.lean>.
- [11] Lean contributors. *Init/Prelude.lean*: the equations defining Nat.add. Source excerpt inspected at revision 43cdd170d0cc1adf3a7774a2328f9e4797a515a3. <https://github.com/leanprover/lean4/blob/43cdd170d0cc1adf3a7774a2328f9e4797a515a3/src/Init/Prelude.lean>.
- [12] Lean contributors. *Lean/LibrarySuggestions/Basic.lean*. Source at tag v4.34.0; Selector, Config, and Suggestion. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/LibrarySuggestions/Basic.lean>.
- [13] Lean project. *The Lean Language Reference: Recursive Definitions*. Rolling reference consulted September 21, 2026. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>.
- [14] Lean project. *The Lean Language Reference: Validating Proofs*. Rolling reference consulted September 21, 2026. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [15] J. Limperg and A. H. From. Aesop: White-Box Best-First Proof Search for Lean. *Certified Programs and Proofs (CPP)*, 2023. <https://doi.org/10.1145/3573105.3575671>.
- [16] S. Barke, H. Peleg, and N. Polikarpova. Just-in-Time Learning for Bottom-Up Enumerative Synthesis. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA), 2020. DOI: 10.1145/3428295. <https://arxiv.org/abs/2010.08663>.
- [17] M. Luby, A. Sinclair, and D. Zuckerman. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters*, 47:173–180, 1993. [https://doi.org/10.1016/0020-0190\(93\)90029-9](https://doi.org/10.1016/0020-0190(93)90029-9).

- [18] J. Gibbons, G. Hutton, and T. Altenkirch. When Is a Function a Fold or an Unfold? *Electronic Notes in Theoretical Computer Science*, 44(1), 2001. Author publication record: <https://www.cs.ox.ac.uk/publications/publication2368-abstract.html>.
- [19] N. Mulleners, J. Jeuring, and B. Heeren. Example-Based Reasoning about the Realizability of Polymorphic Programs. *Proceedings of the ACM on Programming Languages*, 8(ICFP), article 247, 2024. DOI: 10.1145/3674636. <https://arxiv.org/abs/2406.18304>.
- [20] R. Ji, Y. Zhao, Y. Xiong, D. Wang, L. Zhang, and Z. Hu. Decomposition-Based Synthesis for Applying Divide-and-Conquer-Like Algorithmic Paradigms. *ACM Transactions on Programming Languages and Systems*, 2024. Inspected author manuscript, arXiv:2202.12193v4, January 29, 2024. <https://arxiv.org/html/2202.12193v4>.
- [21] J. Lubin, N. Collins, C. Omar, and R. Chugh. Program Sketching with Live Bidirectional Evaluation. *Proceedings of the ACM on Programming Languages*, 4(ICFP), article 109, 2020. DOI: 10.1145/3408991. <https://arxiv.org/abs/1911.00583>.
- [22] M. A. Hammer, Y. P. Khoo, M. Hicks, and J. S. Foster. Adapton: Composable, Demand-Driven Incremental Computation. *Programming Language Design and Implementation (PLDI)*, 2014. Author artifact and paper page: <https://www.cs.umd.edu/projects/PL/adapton/>.
- [23] M. Carneiro. Lean4Lean: Towards a Verified Typechecker for Lean, in Lean. 2024. Author manuscript, arXiv:2403.14064. <https://arxiv.org/abs/2403.14064>.
- [24] M. Carneiro. Lean4Lean: Mechanizing the Metatheory of Lean. *Workshop on the Implementation of Type Systems (WITS)*, January 17, 2026. Invited presentation and project-status abstract. <https://popl26.sigplan.org/details/wits-2026-papers/11/-Lean4Lean-Mechanizing-the-Metatheory-of-Lean>.
- [25] A. Miltner, A. Trejo Nuñez, A. Brendel, S. Chaudhuri, and I. Dillig. Bottom-up Synthesis of Recursive Functional Programs using Angelic Execution. *Proceedings of the ACM on Programming Languages*, 6(POPL), article 21, 2022. DOI: 10.1145/3498682. <https://arxiv.org/abs/2107.06253>.
- [26] C. Norman and J. Avigad. Canonical for Automated Theorem Proving in Lean. *Interactive Theorem Proving (ITP)*, LIPIcs 352, 14:1–14:20, 2025. DOI: 10.4230/LIPIcs.ITP.2025.14. Technical examples inspected in the first author-manuscript version: <https://arxiv.org/html/2504.06239v1>. Publication and updated manuscript record: <https://arxiv.org/abs/2504.06239>.
- [27] Ł. Czakajka. Practical Proof Search for Coq by Type Inhabitation. *International Joint Conference on Automated Reasoning (IJCAR)*, 2020. Open-access full text: <https://pmc.ncbi.nlm.nih.gov/articles/PMC7324019/>.
- [28] CoqHammer project. *CoqHammer and sauto documentation*. Consulted September 21, 2026; unfolding options and limits of reconstruction tactics. <https://coqhammer.github.io/>.
- [29] N. Polikarpova, I. Kuraj, and A. Solar-Lezama. Program Synthesis from Polymorphic Refinement Types. *Programming Language Design and Implementation (PLDI)*, 2016. DOI: 10.1145/2908080.2908093. <https://arxiv.org/abs/1510.08419>.
- [30] Z. Guo, M. James, D. Justo, J. Zhou, Z. Wang, R. Jhala, and N. Polikarpova. Program Synthesis by Type-Guided Abstraction Refinement. *Proceedings of the ACM on Programming Languages*, 4(POPL), 2020. DOI: 10.1145/3371080. <https://arxiv.org/abs/1911.04091>.
- [31] T. Zhu, J. Clune, J. Avigad, A. Q. Jiang, and S. Welleck. Premise Selection for a Lean Hammer. *International Conference on Learning Representations (ICLR)*, 2026. Author manuscript, arXiv:2506.07477v2, February 25, 2026. <https://arxiv.org/abs/2506.07477>.